

Modelos Generativos Profundos

Clase 6: Modelos autorregresivos y LLMs (parte II)

Fernando Fêtis Riquelme

Primavera, 2025

Facultad de Ciencias Físicas y Matemáticas

Universidad de Chile

Recuerdo clase anterior

Arquitectura GPT

Recuerdo clase anterior

Modelos autorregresivos

Dada una secuencia de **tokens**, $S = (w_1, \dots, w_T) \in \mathcal{V}^*$ sobre un **vocabulario** $\mathcal{V}$, un **LM** factoriza $p_\theta(S)$ de forma autorregresiva:

$$p_\theta(w_1, \dots, w_T) = p_\theta(w_1) \prod_{t=1}^{T-1} p_\theta(w_{t+1} | w_1, \dots, w_t),$$

donde $p_\theta(w_{t+1} | w_1, \dots, w_t) \sim \text{Categ\acute{o}rica}(\mathcal{V}; \lambda_\theta(w_1, \dots, w_t))$.

El vector de probabilidades $\lambda_\theta(w_1, \dots, w_t) \in \Delta^{|\mathcal{V}|}$ es aprendido por una red neuronal $\lambda_\theta : \mathcal{V}^* \rightarrow [0, 1]^{|\mathcal{V}|}$ cuya entrada es de longitud variable.

Esto \u00faltimo motiv\u00f3 a utilizar una **RNN** para modelar λ_θ .

Modelos autorregresivos

Dado un conjunto de entrenamiento $\mathcal{D} = \{S^1, \dots, S^N\} \subset \mathcal{V}^*$, la red neuronal λ_θ se entrena optimizando

$$\max_{\theta} \frac{1}{|\mathcal{D}|} \sum_{S \in \mathcal{D}} \frac{1}{|S|} \log p_{\theta}(S),$$

con $\log p_{\theta}(S) = \sum_{t=1}^{T-1} \log p_{\theta}(w_{t+1} | w_1, \dots, w_t)$ si $S = (w_1, \dots, w_T)$.

Para la generación, se comienza con $w_1 = \langle \text{BOS} \rangle \in \mathcal{V}$, y predice el siguiente token usando el LM. Esto se repite hasta generar el token $\langle \text{EOS} \rangle \in \mathcal{V}$ o hasta alcanzar una longitud máxima.

Arquitectura GPT

Motivación

Las RNNs tienen limitaciones:

- Entrenamiento secuencial no paralelizable.
- Dificultad para retener información a largo plazo.
- Sesgos hacia el final de la secuencia.

La **arquitectura GPT** soluciona, al menos en principio, estas limitaciones:

- Permite procesar una secuencia completa de forma paralela.
- En todo momento es posible acceder a la información contenida en cualquier parte de la secuencia.
- No hay posiciones privilegiadas en la secuencia.

La arquitectura GPT está compuesta por **bloques Transformer**, cada uno de los cuales tiene dos componentes principales:

- Un mecanismo de **self-attention** que permite, para cada token de la secuencia, obtener embeddings enriquecidos con información contextual de los otros tokens de la secuencia.
- Una **red feedforward** que procesa cada embedding contextual de forma individual para obtener un embedding final.

Esta arquitectura también incluye elementos adicionales como **positional encodings**, **capas de normalización** y **dropout**.

A grandes rasgos, la arquitectura GPT aplica los siguientes componentes en el orden indicado:

1. Capa de token embedding (con positional encoding).
2. Bloques Transformers:
 - Self-attention (con layer norm y conexión residual).
 - Red feedforward (con layer norm y conexión residual).
3. Cabezal de lenguaje.

Cada bloque Transformer puede verse como un módulo usado para *mejorar* los embeddings iniciales de la secuencia.

Layer normalization

La aplicación consecutiva de varias capas neuronales suele afectar el entrenamiento de la red (p.g. por **vanishing gradient** o **exploding gradient**).

- Este problema se puede aminorar utilizando **capas de normalización** entre los bloques de la red.
- En redes convolucionales es usual utilizar **batch normalization** o **group normalization**.
- No es obvio cómo aplicar batch norm en modelos secuenciales: las secuencias de tokens no suelen tener el mismo largo y muchas veces los batches de entrenamiento deben ser pequeños.

Layer normalization

Para una secuencia de embeddings, **layer normalization** normaliza cada vector de embedding de forma individual.

Para un vector de embedding de la secuencia, $x \in \mathbb{R}^D$, primero se obtiene el embedding normalizado $\tilde{x} \in \mathbb{R}^D$, donde:

$$\tilde{x}_d = \frac{x_d - \mu}{\sigma}, \quad \text{para} \quad \mu = \frac{1}{D} \sum_{d=1}^D x_d \quad \text{y} \quad \sigma^2 = \frac{1}{D} \sum_{d=1}^D (x_d - \mu)^2$$

Luego, de forma análoga a lo realizado por batch norm, layer norm aplica una transformación coordenada a coordenada:

$$\text{LayerNorm}(x) = \alpha \odot \tilde{x} + \beta,$$

donde $\alpha, \beta \in \mathbb{R}^D$ son parámetros entrenables.

Self-attention

Dada una secuencia de embeddings, $(x_1, \dots, x_T) \subset \mathbb{R}^D$:

- El objetivo del **mecanismo de self-attention** es obtener una secuencia de *embeddings enriquecidos*, $(y_1, \dots, y_T) \subset \mathbb{R}^P$.
- Cada **embedding contextual** $y_t \in \mathbb{R}^P$ combina el embedding $x_t \in \mathbb{R}^D$ con todos los otros vectores de embedding, $(x_1, \dots, x_T)$, para poder incorporar información contextual.
- Esto soluciona el problema de memoria a largo plazo presente en las redes recurrentes.

Self-attention

- Se comienza proyectando cada vector de embeddings de la secuencia $(x_1, \dots, x_T) \subset \mathbb{R}^D$ a $\mathbb{R}^P$. Se necesitan 3 proyecciones:

$$q_t = W_Q x_t, \quad k_t = W_K x_t, \quad v_t = W_V x_t, \quad \forall t \in \{1, \dots, T\}.$$

- $W_Q, W_K, W_V \in \mathcal{M}_{P,D}(\mathbb{R})$ son parámetros entrenables.
- Cada embedding enriquecido, $y_i \in \mathbb{R}^P$, se calcula como

$$y_i = \sum_{j=1}^T a_{ij} v_j, \quad \text{donde} \quad a_{ij} \geq 0 \text{ y } \sum_{j=1}^T a_{ij} = 1$$

- $a_{ij} \in [0, 1]$ representa la **atención** que coloca el token i al token j . La elección usual es la **scaled dot-product attention**:

$$a_{ij} = \text{Softmax}\{s_{i1}, \dots, s_{iT}\}_j = \frac{e^{s_{ij}}}{\sum_{t=1}^T e^{s_{it}}}, \quad \text{donde} \quad s_{ij} = \frac{\langle q_i, k_j \rangle}{\sqrt{P}}$$

Self-attention

Para paralelizar el cálculo de la secuencia $(y_1, \dots, y_T) \subset \mathbb{R}^P$ se deben guardar las proyecciones en matrices $Q, K, V \in \mathcal{M}_{T,P}(\mathbb{R})$:

$$Q = \begin{pmatrix} q_1^\top \\ \vdots \\ q_T^\top \end{pmatrix}, \quad K = \begin{pmatrix} k_1^\top \\ \vdots \\ k_T^\top \end{pmatrix}, \quad V = \begin{pmatrix} v_1^\top \\ \vdots \\ v_T^\top \end{pmatrix}$$

Dado que $(QK^\top)_{ij} = \langle q_i, k_j \rangle$, entonces $s_{ij} = \left(\frac{QK^\top}{\sqrt{P}} \right)_{ij}$.

Notando que $A = \text{softmax}_{\text{por filas}}(S) \in \mathcal{M}_{T,T}(\mathbb{R})$, se tiene que:

$$\begin{pmatrix} y_1^\top \\ \vdots \\ y_T^\top \end{pmatrix} = \text{softmax}_{\text{por filas}} \left(\frac{QK^\top}{\sqrt{P}} \right) V \in \mathcal{M}_{T,P}(\mathbb{R})$$

Masked self-attention

El mecanismo anterior pierde la **causalidad** del lenguaje al momento de computar los embeddings $(y_1, \dots, y_T) \subset \mathbb{R}^P$:

- Cada embedding $y_t \in \mathbb{R}^P$ solo debería poder acceder a los embeddings $x_1, \dots, x_t$ y no a los embeddings futuros, $x_{t+1}, \dots, x_T$.
- Para imponer esto, se pueden **enmascarar** los tokens futuros forzando que $a_{ij} = 0$ para $j > i$.

Sin embargo, no se puede simplemente sustituir $a_{ij} = 0$ para $j > i$ ya que se dejaría de cumplir la condición $\sum_{j=1}^T a_{ij} = 1$.

Lo usual es modificar $S \in \mathcal{M}_{T,T}(\mathbb{R})$, sustituyendo cada entrada s_{ij} (con $j > i$) por un valor muy negativo, de manera que $a_{ij} \approx 0$.

Multihead self-attention

En las redes convolucionales es usual utilizar varios filtros convolucionales para capturar diferentes patrones.

De forma análoga, en el mecanismo de self-attention se pueden utilizar múltiples **cabezales de atención** (MHSA) para capturar diferentes relaciones entre los tokens:

- Si $(x_1, \dots, x_T) \subset \mathbb{R}^D$ es una secuencia de vectores de embeddings e $(y_1^{(h)}, \dots, y_T^{(h)}) \subset \mathbb{R}^P$ es la salida del h -ésimo cabezal de atención, entonces se puede considerar embeddings

$$\tilde{y}_t = \left(y_t^{(1)} \cdots y_t^{(H)} \right) \in \mathbb{R}^{P \cdot H}, \quad \text{para } t \in \{1, \dots, T\}$$

- Los embeddings finales $(u_1, \dots, u_T)$ que retorna un MHSA son de la forma $u_t = W_0 \tilde{y}_t \in \mathbb{R}^D$, donde la matriz $W_0 \in \mathcal{M}_{D, P \cdot H}(\mathbb{R})$ proyecta los embeddings $(\tilde{y}_1, \dots, \tilde{y}_T)$ de vuelta al espacio $\mathbb{R}^D$.

Red feedforward

El último paso que realiza un bloque Transformer es pasar cada uno de los embeddings contextuales, $(u_1, \dots, u_T) \subset \mathbb{R}^D$, por una misma MLP de dos capas. Para $t \in \{1, \dots, T\}$:

$$\text{FF}(u_t) = W_2 \phi(W_1 u_t), \quad \text{donde:}$$

- $W_1 \in \mathcal{M}_{J,D}(\mathbb{R})$, $W_2 \in \mathcal{M}_{D,J}(\mathbb{R})$ son los parámetros de la red.
- $\phi : \mathbb{R} \rightarrow \mathbb{R}$ es una función de activación que se aplica coordenada a coordenada (p.g., ReLU o GELU).
- Es usual considerar $J \gg D$ para darle más **capacidad** a la red.

Bloque Transformer

Almacenando la secuencia de embeddings, $(x_1, \dots, x_T) \subset \mathbb{R}^D$, en las filas de una matriz $X \in \mathcal{M}_{T,D}(\mathbb{R})$, un **bloque Transformer** tiene la siguiente estructura:

$$\begin{cases} Y = X + \text{Dropout}_p(\text{MHSA}(\text{LayerNorm}(X))) \\ \text{TransformerBlock}(X) = Y + \text{Dropout}_p(\text{FF}(\text{LayerNorm}(Y))) \end{cases}$$

- Las capas de LN y la red FF se aplican de forma individual sobre cada vector de embedding (i.e., en cada fila de la matriz).
- Recordar que, para $x \in \mathbb{R}^D$, $\text{Dropout}_p(x) := \frac{1}{1-p} m \odot x$, donde la máscara $m \in \{0, 1\}^D$ es un vector con $m_d \sim \text{Bernoulli}(1 - p)$.

Arquitectura GPT

Con esto, la arquitectura GPT realiza lo siguiente:

- Recibe una secuencia de tokens de entrada, $(w_1, \dots, w_T) \in \mathcal{V}^*$.
- Esta secuencia es pasada por una capa de token embedding.
- Para inyectar la información posicional de cada token, se suma un vector de **positional embedding** a cada token embedding.
- Esta suma define los embeddings iniciales, los cuales luego son pasados por varios bloques Transformer consecutivos.
- Finalmente, se agrega un cabezal de lenguaje para proyectar los embedding finales a $\Delta^{|\mathcal{V}|}$.

En particular, se puede utilizar la salida del último token para predecir el siguiente token de la secuencia.

En la próxima clase:

- Variaciones de la arquitectura GPT.

Modelos Generativos Profundos

Clase 6: Modelos autorregresivos y LLMs (parte II)